NumPy paso a paso

Cómo leer, resolver y probar el Homework 1 (Session 2) y cómo abordar problemas parecidos con máscaras booleanas

1. **Leer el problema**

   Traducir el enunciado en inglés a reglas concretas.

2. **Recorrer el notebook**

   Qué hace cada celda y cuáles se tocan.

3. **Resolver**

   La máscara booleana, paso por paso, con colores.

4. **Probar y entregar**

   Casos de prueba, casos borde y Omnicampus.

5. **Método general**

   Una receta de 5 pasos para problemas similares.

6. **Problemas resueltos**

   Ocho variaciones con su código y salida real.

7. **Caja de herramientas**

   Las funciones de NumPy que más se usan.

8. **Practicar**

   Ejercicios con respuestas al final.

Generado el 7 de octubre de 2026 · NumPy 2.5.3 · Todas las salidas de código fueron ejecutadas al crear este documento.

Entrega del Homework 1: jueves 8 de octubre de 2026, 11:00 UTC (6:00 a.m. hora de Colombia).

Antes de empezar: preparar el entorno

Un **entorno virtual** es una carpeta con su propio Python y sus propias librerías. Así cada curso tiene sus versiones sin afectar al resto del computador. El del curso está en la carpeta `03.Lectura Materials & HomeWork/.venv`.

Paso	Comando en la terminal	Cuándo
1. Entrar a la carpeta	`cd "/Users/nato/Downloads/GCI Word_202609/03.Lectura Materials & HomeWork/Session2"`	Siempre
2. Crear el entorno	`python3 -m venv ../.venv`	Una sola vez (ya está hecho)
3. Activarlo	`source ../.venv/bin/activate`	Cada terminal nueva
4. Instalar NumPy	`pip install numpy ipykernel`	Una sola vez (ya está hecho)
5. Verificar	`python -c "import numpy as np; print(np.__version__)"`	Cuando quieras
6. Salir	`deactivate`	Al terminar

Cómo saber que está activo

La línea de la terminal empieza con `(.venv)`. En VS Code, para usarlo en el notebook: **Select Kernel** (arriba a la derecha) → **Python Environments** → **.venv**.

Por qué las comillas en la ruta

La carpeta tiene espacios y el símbolo `&`. Sin comillas la terminal la corta en pedazos y no la encuentra.

1. Leer el problema

El error más común no está en el código sino en la lectura. Antes de escribir nada, hay que responder tres preguntas: **qué entra**, **qué sale** y **qué regla** decide qué sale.

> ENUNCIADO ORIGINAL
> *Implement a function that takes a 1D NumPy array of integers and returns an array containing elements that are multiples of 5 and leave a remainder of 1 when divided by 2.*

Frase por frase

Frase del enunciado	Qué significa	En código
takes a 1D NumPy array of integers	Entra un arreglo de una dimensión con enteros	`a`
returns an array containing elements…	Sale un arreglo con **algunos** de los elementos: es un filtro	`a[mask]`
multiples of 5	Al dividir por 5 no sobra nada	`a % 5 == 0`
and	Las dos reglas deben cumplirse a la vez	`&`
leave a remainder of 1 when divided by 2	Al dividir por 2 sobra 1, es decir, es impar	`a % 2 == 1`

Las tres preguntas, respondidas

Pregunta	Respuesta para el HW1
¿Qué entra?	Un arreglo 1D de enteros, por ejemplo `[1, 5, 10, 3, 4, 25, 30]`
¿Qué sale?	Un arreglo más corto, solo con los elementos que cumplen. Puede quedar vacío.
¿Qué regla?	Múltiplo de 5 **y** impar.

Comprobar a mano antes de programar

Resolver el ejemplo con lápiz confirma que se entendió la regla. Si a mano no da `[5 25]`, el código tampoco lo dará.

Número	¿Múltiplo de 5?	¿Impar?	¿Se queda?
1	No	Sí	No
5	Sí	Sí	**Sí**
10	Sí	No	No
3	No	Sí	No
4	No	No	No
25	Sí	Sí	**Sí**
30	Sí	No	No

Diccionario: frases típicas de enunciados → código NumPy

En inglés	En español	En NumPy
multiple of n / divisible by n	múltiplo de n	`a % n == 0`
even / odd	par / impar	`a % 2 == 0 / a % 2 == 1`
remainder r when divided by n	residuo r al dividir por n	`a % n == r`
greater than / at least	mayor que / al menos	`a > x / a >= x`
less than / at most	menor que / como máximo	`a < x / a <= x`
between x and y (inclusive)	entre x y y (incluidos)	`(a >= x) & (a <= y)`
and / or / not	y / o / no	`& / | / ~`
positive / negative	positivo / negativo	`a > 0 / a < 0`
count / how many	contar / cuántos	`np.count_nonzero(mask)`
sum / average	sumar / promedio	`a[mask].sum() / a.mean()`
replace … with …	reemplazar … por …	`np.where(mask, nuevo, a)`
indices / positions	posiciones	`np.where(mask)[0]`
unique / sorted	sin repetidos / ordenado	`np.unique(a) / np.sort(a)`
rows / columns	filas / columnas	`axis=1 / axis=0`

2. Recorrido por las celdas del notebook

El notebook `HW1 for Session2.ipynb` tiene 25 celdas. Solo **una** se edita; el resto se lee o se ejecuta. Las celdas se ejecutan **de arriba hacia abajo** con **Shift + Enter**.

Celda	Qué contiene	Qué haces
0	Título, fecha límite y flujo recomendado	Leer
1	La pregunta: enunciado, entradas, salidas y ejemplo	Leer
2	Sección 1. Preparation	Leer
3	import numpy as np → carga NumPy con el alias np	Ejecutar
4	a = np.array([1, 5, 10, 3, 4, 25, 30]) → crea el arreglo de ejemplo	Ejecutar
5	Sección 2. Your Answer: reglas (sin imports dentro de la función)	Leer
6	def homework(a): … → aquí va tu solución	EDITAR
7–8	Sección 3. Test Your Answer	Leer
9	Caso 1: homework(a) con el ejemplo	Ejecutar
10	Resultado esperado del caso 1: [5 25]	Comparar
11–13	Caso 2 con [11, 15, 20, 21, 35, 40, 45] → esperado [15 35 45]	Ejecutar
14–16	Caso 3 con [2, 4, 6, 8] → esperado [] (vacío)	Ejecutar
17	Checklist final antes de entregar	Leer
18–23	Sección 4. Cómo entregar en Omnicampus (con capturas)	Seguir
24	Reglas de calificación: cuenta la última entrega	Leer

La celda 6 antes y después

El notebook original trae un error a propósito: la línea `some` no existe en Python y produce `NameError`. Hay que borrar el contenido completo de la celda y escribir la solución.

ANTES (ORIGINAL)

```
def homework(a):
    # Write your answer here
    some
    my_result = None
    return my_result
```

DESPUÉS (SOLUCIÓN)

```
def homework(a):
    mask = (a % 5 == 0) & (a % 2 == 1)
    my_result = a[mask]
    return my_result
```

> **Si cambias la función, vuelve a ejecutar su celda**
>
> Jupyter recuerda la última versión ejecutada, no la que ves escrita. Después de editar la celda 6, ejecútala de nuevo y luego las pruebas.

3. Resolver: la máscara booleana

NumPy no revisa los números uno por uno con un ciclo. Hace la misma pregunta a **todo el arreglo a la vez** y responde con otro arreglo del mismo tamaño lleno de `True` y `False`. A ese arreglo se le llama **máscara**.

PASO 1 · EL RESIDUO DE DIVIDIR POR 5

```
import numpy as np
a = np.array([1, 5, 10, 3, 4, 25, 30])
print(a % 5)
```

SALIDA REAL

```
[1 0 0 3 4 0 0]
```

PASO 2 · CONVERTIR CADA REGLA EN TRUE / FALSE

```
print(a % 5 == 0)
print(a % 2 == 1)
```

SALIDA REAL

```
[False  True  True False False  True  True]
[ True  True False  True False  True False]
```

a	1	5	10	3	4	25	30
`a % 5`	1	0	0	3	4	0	0
`a % 5 == 0`	**False**	**True**	**True**	**False**	**False**	**True**	**True**
`a % 2`	1	1	0	1	0	1	0
`a % 2 == 1`	**True**	**True**	**False**	**True**	**False**	**True**	**False**
`(…) & (…) → mask`	**False**	**True**	**False**	**False**	**False**	**True**	**False**

Verde = True, rojo = False. La última fila solo es verde donde las dos filas de arriba son verdes.

PASO 3 · COMBINAR CON & ("Y")

```
mask = (a % 5 == 0) & (a % 2 == 1)
print(mask)
```

SALIDA REAL

```
[False  True False False False  True False]
```

PASO 4 · FILTRAR: QUEDARSE CON LOS TRUE

```
print(a[mask])
```

SALIDA REAL

```
[ 5 25]
```

La idea completa en una frase

Crear la máscara con condiciones y **usarla como filtro** con `a[mask]`. Casi todos los problemas de esta sesión son variaciones de eso.

Los paréntesis son obligatorios

En Python, `&` se evalúa antes que `==`. Sin paréntesis, `a % 5 == 0 & a % 2 == 1` se interpreta como `a % 5 == (0 & a % 2) == 1`, que no es lo que queremos.

4. Probar y entregar

LA SOLUCIÓN Y SUS PRUEBAS

```
def homework(a):
    mask = (a % 5 == 0) & (a % 2 == 1)
    my_result = a[mask]
    return my_result

pruebas = [
    ("Caso 1",    [1, 5, 10, 3, 4, 25, 30], [5, 25]),
    ("Caso 2",    [11, 15, 20, 21, 35, 40, 45], [15, 35, 45]),
    ("Caso 3",    [2, 4, 6, 8], []),
    ("Negativos", [-5, -15, -10, -25, 0, 7], [-5, -15, -25]),
    ("Vacío",     [], []),
]
for nombre, entrada, esperado in pruebas:
    r = homework(np.array(entrada, dtype=int))
    ok = np.array_equal(r, esperado)
    print(f"{nombre:10} {str(r):14} {'OK' if ok else 'FALLA'}")
```

SALIDA REAL

```
Caso 1     [ 5 25]        OK
Caso 2     [15 35 45]     OK
Caso 3     []             OK
Negativos  [ -5 -15 -25]  OK
Vacío      []             OK
```

Los casos **Negativos** y **Vacío** no están en el notebook, pero el evaluador de Omnicampus usa casos ocultos de ese tipo. En Python `-5 % 2` da `1`, por eso los negativos impares se detectan bien.

Entregar en Omnicampus

1. Copiar la función completa, desde `def homework(a):` hasta `return my_result`.
2. No copiar `import numpy` ni las celdas de prueba: el evaluador ya incluye NumPy.
3. Pegar en la tarea y hacer clic en **[Python code submission]**.
4. Esperar la nota. **3.0** es correcto. Cuenta la **última** entrega: no reenviar después de un 3.0. Una entrega tardía correcta vale máximo 2.0.

5. Método general para problemas similares

Este método sirve para cualquier ejercicio que pida filtrar, contar, sumar o reemplazar elementos de un arreglo.

1 **Leer**

Responder: ¿qué entra?, ¿qué sale?, ¿qué regla? Subrayar palabras clave: *multiple, even, odd, greater, and, or, not, count, sum, replace*.

2 **Traducir**

Convertir cada regla en una comparación: `a % 5 == 0`, `a > 10`, `a < 0`…

3 **Combinar**

Unir reglas con `&` (y), `|` (o), `~` (no). Cada regla entre paréntesis. El resultado es la máscara.

4 **Aplicar**

Elegir la herramienta según lo que pide la salida (tabla de abajo).

5 **Probar**

Resolver el ejemplo a mano, imprimir la máscara y probar casos borde: vacío, negativos, cero, ninguno cumple, todos cumplen.

Paso 4: ¿qué pide la salida?

Si el enunciado pide…	La salida es…	Herramienta
los elementos que cumplen	un arreglo más corto	`a[mask]`
cuántos cumplen	un número entero	`np.count_nonzero(mask)`
la suma / el promedio de los que cumplen	un número	`a[mask].sum() / a[mask].mean()`
reemplazar los que cumplen	un arreglo del mismo tamaño	`np.where(mask, nuevo, a)`
las posiciones de los que cumplen	un arreglo de índices	`np.where(mask)[0]`
si alguno / si todos cumplen	True o False	`mask.any() / mask.all()`

Truco para depurar

Cuando el resultado no es el esperado, imprime cada paso por separado: `print(a % 5)`, `print(mask)`. La fila que no coincide con tu cálculo a mano es donde está el error.

Revisa también `a.shape` y `a.dtype` para confirmar la forma y el tipo de los datos.

6. Problemas similares, resueltos

Cada problema aplica el método: regla → máscara → herramienta. Las salidas son reales.

A. Pares mayores que 10

Return the elements that are even and greater than 10.

Idea: Dos reglas con **y**; la salida es un filtro.

```
a = np.array([4, 12, 15, 18, 7, 22, 10])
mask = (a % 2 == 0) & (a > 10)
print(a[mask])
```

SALIDA REAL

```
[12 18 22]
```

B. Múltiplos de 3 o de 7

Return the elements that are multiples of 3 or multiples of 7.

Idea: La palabra **or** se traduce como `|`.

```
a = np.array([1, 3, 7, 10, 14, 15, 20, 21])
mask = (a % 3 == 0) | (a % 7 == 0)
print(a[mask])
```

SALIDA REAL

```
[ 3  7 14 15 21]
```

C. Contar en lugar de filtrar

Return how many elements are odd multiples of 5.

Idea: La misma máscara del HW1, pero la salida es un número.

```
a = np.array([1, 5, 10, 3, 4, 25, 30])
mask = (a % 5 == 0) & (a % 2 == 1)
print(np.count_nonzero(mask))
```

SALIDA REAL

```
2
```

D. Sumar los negativos

Return the sum of the negative elements.

Idea: Filtrar y luego reducir con `.sum()`.

```
a = np.array([3, -2, 8, -7, 0, -1])
print(a[a < 0].sum())
```

SALIDA REAL

```
-10
```

E. Reemplazar valores

Replace multiples of 5 with 0 and keep the rest.

Idea: La salida tiene el mismo tamaño: `np.where` elige entre dos valores.

```
a = np.array([1, 5, 10, 3, 4, 25, 30])
print(np.where(a % 5 == 0, 0, a))
```

SALIDA REAL

```
[1 0 0 3 4 0 0]
```

F. Posiciones

Return the indices of the elements greater than the mean.

Idea: Se piden posiciones, no valores: `np.where(mask)[0]`.

```
a = np.array([2, 9, 4, 11, 5])
print(a.mean())
print(np.where(a > a.mean())[0])
```

SALIDA REAL

```
6.2
[1 3]
```

G. Negar una condición

Return the elements that are NOT multiples of 3.

Idea: `~` invierte la máscara: los True pasan a False y viceversa.

```
a = np.array([3, 4, 6, 7, 9, 10])
print(a[~(a % 3 == 0)])
```

SALIDA REAL

```
[ 4  7 10]
```

H. Matriz (2D)

Return the sum of each row and the values between 3 and 7.

Idea: `axis=1` opera fila por fila. Filtrar una matriz devuelve un arreglo 1D.

```
m = np.array([[1, 5, 8],
              [3, 6, 2],
              [7, 4, 9]])
print(m.sum(axis=1))
print(m[(m >= 3) & (m <= 7)])
```

SALIDA REAL

```
[14 11 20]
[5 3 6 7 4]
```

7. Caja de herramientas de NumPy

Las funciones más útiles para este tipo de ejercicios. La columna **Resultado** se calculó ejecutando cada ejemplo con `a = np.array([4, -2, 7, 0, 7, 10])` y `m = np.array([[1, 2, 3], [4, 5, 6]])`.

Crear arreglos

Código	Para qué sirve	Resultado
`np.array([1, 2, 3])`	Crear un arreglo desde una lista	`[1 2 3]`
`np.arange(0, 10, 2)`	Números de 0 a 10 (sin incluir) de 2 en 2	`[0 2 4 6 8]`
`np.zeros(3)`	Arreglo de ceros	`[0. 0. 0.]`
`np.arange(6).reshape(2, 3)`	Cambiar la forma a 2 filas × 3 columnas	`[[0 1 2] [3 4 5]]`

Inspeccionar

Código	Para qué sirve	Resultado
`a.shape`	Forma: cuántos elementos por dimensión	`6`
`a.size`	Cantidad total de elementos	`6`
`a.dtype`	Tipo de dato	`int64`
`m.ndim`	Número de dimensiones	`2`

Operar elemento a elemento

Código	Para qué sirve	Resultado
`a * 2`	Multiplicar todos	`[ 8 -4 14 0 14 20]`
`a % 3`	Residuo de cada uno	`[1 1 1 0 1 1]`
`a // 3`	División entera de cada uno	`[ 1 -1 2 0 2 3]`
`np.abs(a)`	Valor absoluto	`[ 4 2 7 0 7 10]`

Comparar y combinar (crean máscaras)

Código	Para qué sirve	Resultado
`a > 4`	Mayor que	`[False False True False True True]`
`(a > 0) & (a < 8)`	Y: ambas condiciones	`[ True False True False True False]`
`(a < 0) | (a > 8)`	O: al menos una	`[False True False False False True]`

Código	Para qué sirve	Resultado
`~(a > 4)`	No: invierte la máscara	`[ True True False True False False]`
`np.isin(a, [0, 7])`	¿Está en esta lista?	`[False False True True True False]`

Seleccionar

Código	Para qué sirve	Resultado
`a[a > 4]`	Filtrar con máscara	`[ 7 7 10]`
`a[0], a[-1]`	Primer y último elemento	`4, 10`
`a[1:4]`	Del índice 1 al 3	`[-2 7 0]`
`np.where(a > 4)[0]`	Posiciones donde se cumple	`[2 4 5]`
`np.where(a > 4, 1, 0)`	Elegir valor según la condición	`[0 0 1 0 1 1]`
`m[1, 2]`	Fila 1, columna 2 de una matriz	`6`
`m[:, 0]`	Toda la columna 0	`[1 4]`

Resumir (reducir a un número)

Código	Para qué sirve	Resultado
`a.sum(), a.mean()`	Suma y promedio	`26, 4.3333`
`a.min(), a.max()`	Mínimo y máximo	`-2, 10`
`a.argmax()`	Posición del máximo	`5`
`np.count_nonzero(a > 4)`	Contar cuántos True	`3`
`(a > 4).any(), (a > 4).all()`	¿Alguno? ¿Todos?	`True, False`
`m.sum(axis=0)`	Suma por columna	`[5 7 9]`
`m.sum(axis=1)`	Suma por fila	`[ 6 15]`

Ordenar y limpiar

Código	Para qué sirve	Resultado
`np.sort(a)`	Ordenar de menor a mayor	`[-2 0 4 7 7 10]`
`np.unique(a)`	Quitar repetidos (y ordenar)	`[-2 0 4 7 10]`
`np.clip(a, 0, 5)`	Recortar al rango [0, 5]	`[4 0 5 0 5 5]`

8. Errores comunes

Error	Qué pasa	Cómo se arregla
Usar `and` / `or` con arreglos	`ValueError: The truth value of an array… is ambiguous`	Usar `&` / `|`
Olvidar paréntesis	Resultado incorrecto o error	`(a % 5 == 0) & (a % 2 == 1)`
Devolver la máscara	Sale `[False True …]` en vez de números	Devolver `a[mask]`
Usar un ciclo `for` con `append`	Funciona, pero es lento y largo	Usar la máscara
`import` dentro de la función	Omnicampus puede rechazarla	Los imports van fuera; el evaluador ya trae NumPy
No volver a ejecutar la celda editada	Jupyter usa la versión anterior	Ejecutar la celda de la función y luego las pruebas
Probar solo el ejemplo	Falla en casos ocultos	Probar negativos, vacío, cero, ninguno cumple

ASÍ SE VE EL ERROR DE USAR AND

```
a = np.array([1, 5, 10])
try:
    a[(a % 5 == 0) and (a % 2 == 1)]
except ValueError as e:
    print("ValueError:", str(e).split(". ")[0])
```

SALIDA REAL

```
ValueError: The truth value of an array with more than one element is ambiguous
```

9. Ejercicios para practicar

Intenta cada uno con el método de 5 pasos antes de mirar las respuestas en la página siguiente.

Arreglo para todos los ejercicios

```
a = np.array([-6, -3, 0, 4, 9, 12, 15, 20, 27])
```

1. Devuelve los elementos que son múltiplos de 3 y positivos.
2. ¿Cuántos elementos son pares?
3. Reemplaza los negativos por 0.
4. Suma los elementos mayores que 10.
5. Devuelve las posiciones de los múltiplos de 9.
6. Devuelve los elementos que NO están entre 0 y 15 (incluidos).

Respuestas

#	Solución	Resultado real
1	`a[(a % 3 == 0) & (a > 0)]`	`[ 9 12 15 27]`
2	`np.count_nonzero(a % 2 == 0)`	`5`
3	`np.where(a < 0, 0, a)`	`[ 0 0 0 4 9 12 15 20 27]`
4	`a[a > 10].sum()`	`74`
5	`np.where(a % 9 == 0)[0]`	`[2 4 8]`
6	`a[~((a >= 0) & (a <= 15))]`	`[-6 -3 20 27]`

Siguiente paso

Cuando estos salgan sin mirar, intenta escribirlos como función `def ejercicio(a):` con `return`, igual que en Omnicampus, y pruébalos con un arreglo vacío y con uno donde ningún elemento cumpla.